



Today's agenda

↳ left view

↳ right view

↳ Top view

↳ bottom view

↳ vertical order

↳ Construct BT from inorder & preorder



AlgoPrep

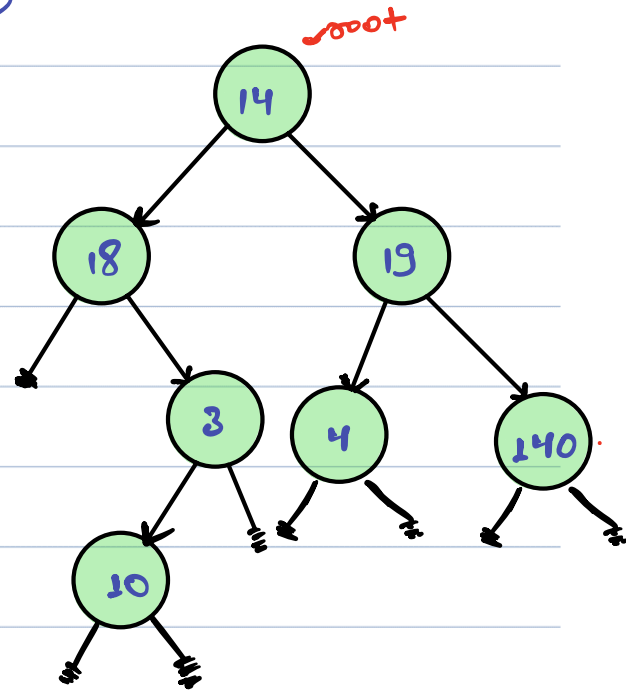


Q) left view

↳ Print Nodes visible from left side.

//idea

↳ Do level order and print the first node of each level.



14 18 3 10

```
void levelOrder (Node root) {
```

```
    Queue < Node > q;
```

```
    q.add(root);
```

```
    while (q.size() > 0) {
```

```
        int n = q.size();
```

```
        for (int i = 1; i <= n; i++) {
```

```
            Node temp = q.remove();
```

```
            if (i == 1) {
```

```
                System.out.print(temp.val);
```

```
            }
```

```
            if (temp.left != null) {
```

```
                q.add(temp.left);
```

```
            }
```

```
            if (temp.right != null) {
```

```
                q.add(temp.right);
```

```
            }
```

```
        }
```

```
        System.out.println();
```

```
    }
```

```
}
```

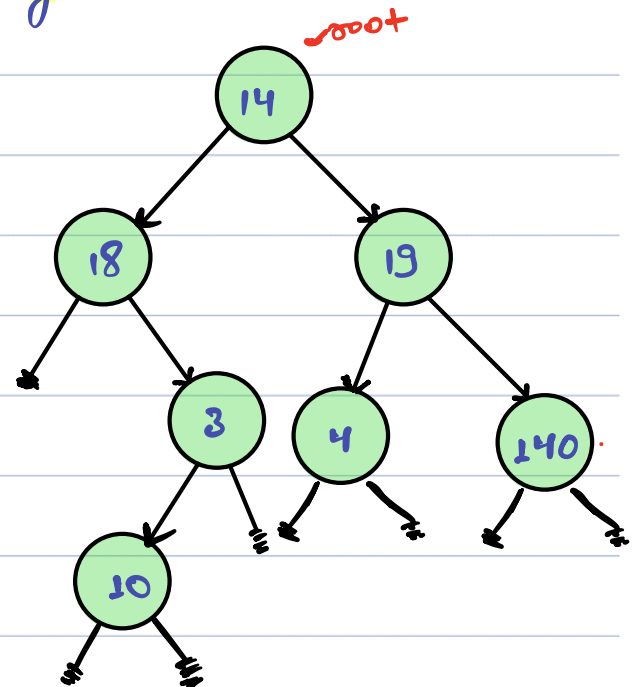


Q) Right view

↳ Point Nodes visible from right side.

Idea:

↳ Point last Node of each level.



```
void levelOrder (Node root) {
```

```
    Queue < Node > q;
```

```
    q.add (root);
```

```
    while (q.size() > 0) {
```

```
        int n = q.size();
```

```
        for (int i = 1; i <= n; i++) {
```

```
            Node temp = q.remove();
```

```
            if (i == n) {
```

```
                System.out.print (temp.val);
```

```
            }
```

```
            if (temp.left != null) {
```

```
                q.add (temp.left);
```

```
            }
```

```
            if (temp.right != null) {
```

```
                q.add (temp.right);
```

```
            }
```

```
        }
```

```
        System.out.println();
```

```
    }
```

T.C: $O(N)$

S.C: $O(N)$

```
class Pair {  
    Node n;  
    int val;  
}
```



Q) Top view

↳ Point Nodes visible from Top side.

<Pair> Q

~~(10, 0)~~ ~~(20, -1)~~ ~~(30, 1)~~
~~(40, -2)~~ ~~(50, 0)~~ ~~(60, 0)~~ ~~(70, 1)~~
~~(80, -1)~~ ~~(90, 1)~~ ~~(100, -2)~~ ~~(110, 2)~~

map

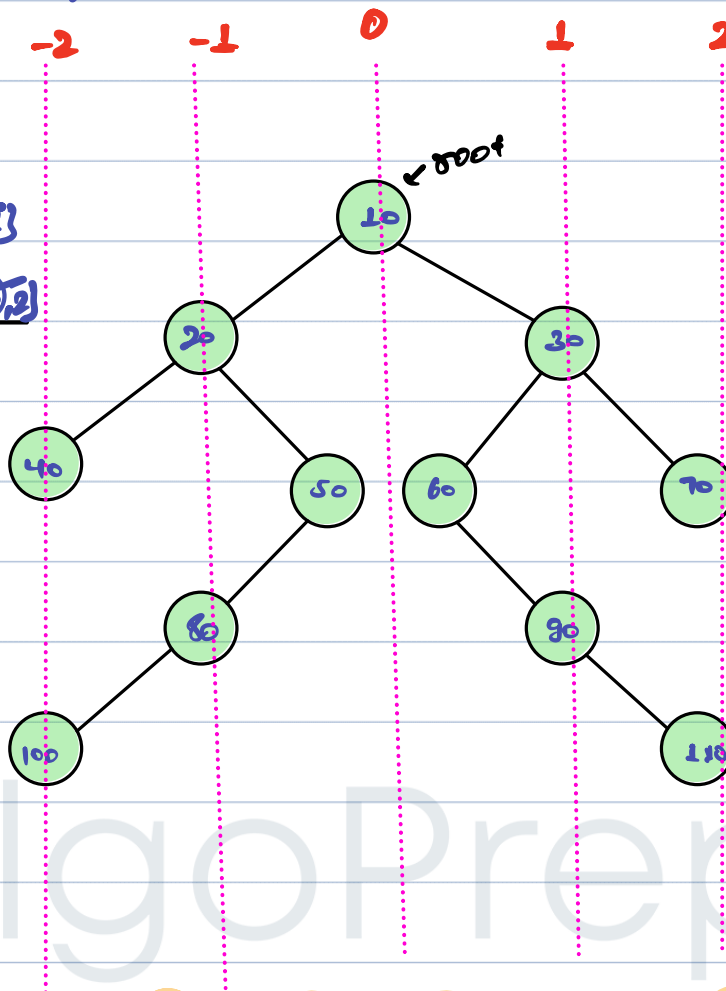
0 → 10

-1 → 20

1 → 30

-2 → 40

2 → 70



→ 40 20 10 30 70

40

20

10

30

70



//Pseudo code

```
Public class Pair {  
    Node n;  
    int vLevel;  
    Pair (Node n1, int n) {  
        n = n1;  
        vLevel = n;  
    }  
}  
  
void toView (Node root) {  
    Queue < Pair > q;  
    HashMap < Integer, Integer > map;  
    q.add (new Pair (root, 0));  
  
    while (q.size() > 0) {  
        Pair rem = q.remove();  
        Node remn = rem.n;  
        int remVL = rem.vLevel;  
  
        if (map.containsKey (remVL) == false) {  
            map.put (remVL, remn.val);  
        }  
  
        if (remn.left != null) {  
            q.add (new Pair (remn.left, remVL + 1));  
        }  
    }  
}
```



```
if (semin.right != null) {  
    q.add(new Pair (semin.right, semin.level+1));  
}
```

//iterate on map and Point

```
q.add (new Pair (root, 0));
```

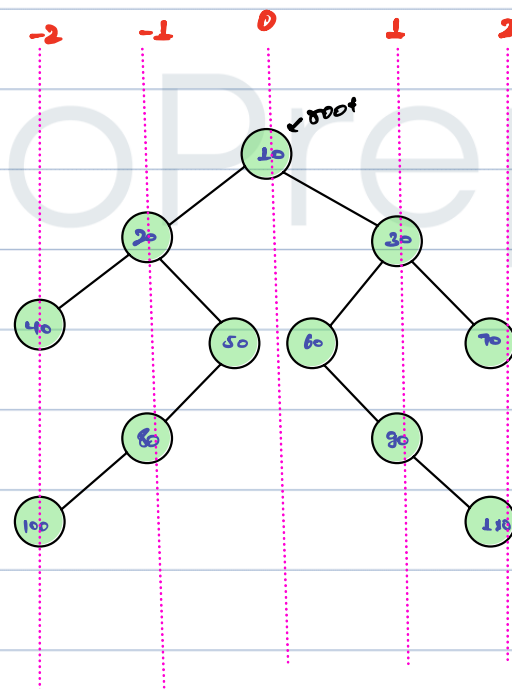
```
while (q.size() > 0) {
```

```
    Pair sem = q.remove();  
    Node semin = sem.n;  
    int semVL = sem.val;
```

```
    if (map.containsKey (semVL) == false) {  
        map.put (semVL, semin.val);  
    }
```

```
    if (semin.left != null) {  
        q.add (new Pair (semin.left, semin.level+1));  
    }
```

```
    if (semin.right != null) {  
        q.add (new Pair (semin.right, semin.level+1));  
    }
```



~~(10, 0)~~ (20, -1) (30, 1)

semin = (40, 0)

semin: 10 semVL: 0

0 → 10



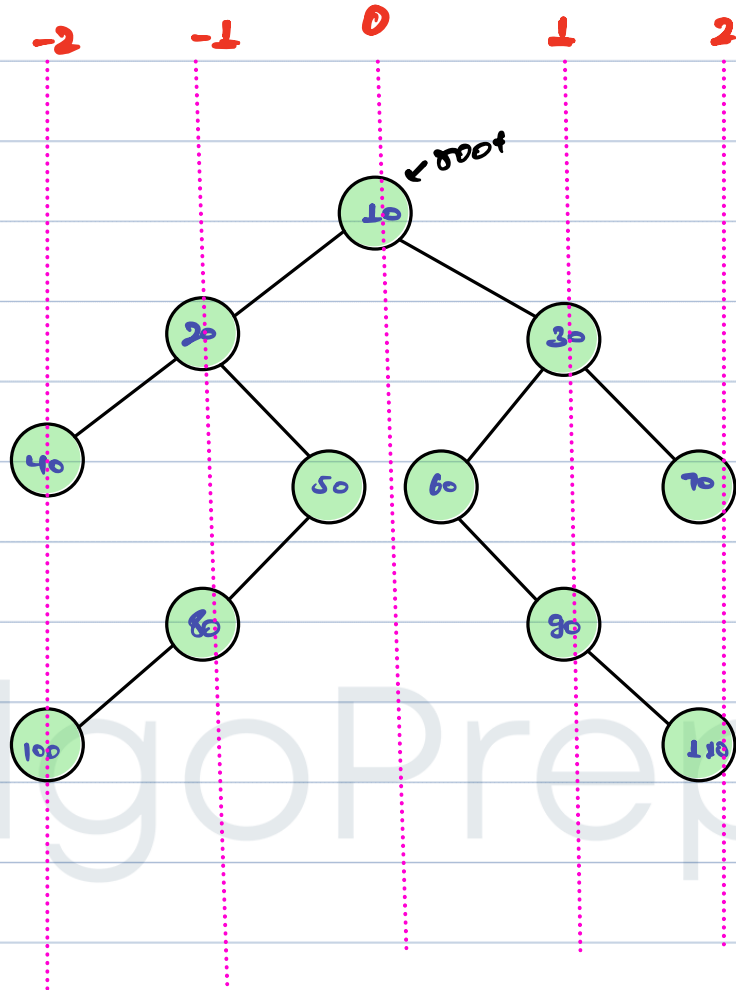
Q) Bottom view

↳ Point Nodes visible from bottom side.

Idea

↳ Point the last node of each vertical level.

↓
Keep updating the Hashmap value.



// Pseudo code

T.C: $O(N)$

S.C: $O(N)$

```
Public class Pair {
```

```
    Node n;
```

```
    int vlevel;
```

```
    Pair (Node n1, int n) {
```

```
        n = n1;
```

```
        vlevel = n;
```

```
    }
```

```
}
```

```
void toView (Node root) {
```

```
    Queue < Pair > q;
```

```
    Hashmap < Integer, Integer > map;
```

q.add(new Pair(root, 0));



while (q.size() > 0) {

Pair rem = q.remove();

Node remn = rem.n;

int remvl = rem.v.level;

~~if (containsKey(remvl) == false) {~~

map.put(remvl, remn.val);

}

if (remn.left != null) {

q.add(new Pair(remn.left, remvl - 1));

}

if (remn.right != null) {

q.add(new Pair(remn.right, remvl + 1));

}

}

// iterate on map and Print



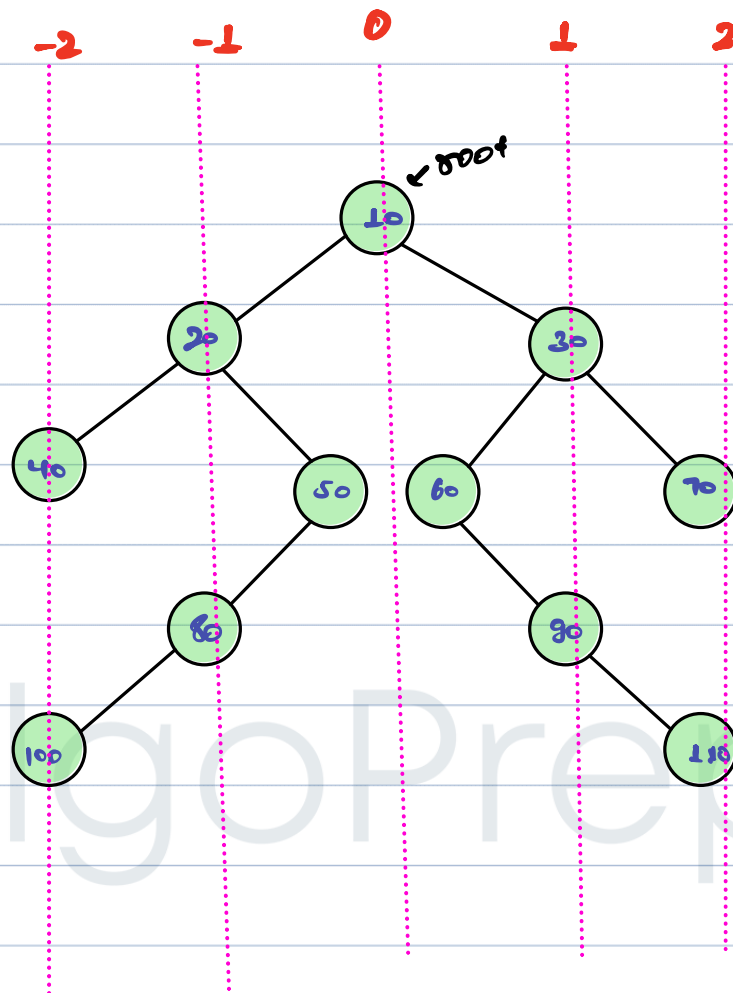
Q) vertical level

↳ Print Nodes vertical level wise.

Integer $\rightarrow$ List<Integer>

0 $\rightarrow$ 10, 50, 60

1 $\rightarrow$



Break till 9:36 AM



Q2) Construction of tree

↳ Given Preorder and inorder of BT of distinct values, generate the trees.

Preorder[]: ^{PS} -11 ¹ -4 ² -3 ³ -6 ⁴ 7 ⁵ -2 ⁶ 3 ^{NR}
Inorder[]: ⁰ -3 ¹ -4 ² -6 ³ -11 ⁴ -2 ⁵ 7 ⁶ 3 ^{NR}

Count: 3

Preorder[]: -4 -3 -6
Inorder[]: -3 -4 -6

Preorder[]: 7 -2 3
Inorder[]: -2 7 3

Preorder[]: -3
Inorder[]: -3

Preorder[]: -6
Inorder[]: -6

Preorder[]: -2
Inorder[]: -2

Preorder[]: 3
Inorder[]: 3

⁰ -3 ¹ -4 ² -6 ³ -11 ⁴ -2 ⁵ 7 ⁶ 3

map

-3 → 0	-2 → 4
-4 → 1	7 → 5
-6 → 2	3 → 6
-11 → 3	



// Pseudo code

```
Node Construct ( int Pre0[], intn-1 ps, int0 pe, int0 in[], int0 is, intn-1 ie ) {  
    if ( ps > pe || is > ie ) { return null; }
```

```
    Node root = new Node ( Pre[ps] );
```

```
        int idn = 0;  
        for ( int i = is; i <= ie; i++ ) {  
            if ( in[i] == Pre[ps] ) {  
                idn = i; break;  
            }  
        }
```

```
        int count = idn - is;
```

```
    root.left = Construct ( Pre, ps+1, ps+count, in, is, idn-1 );  
    root.right = Construct ( Pre, ps+count+1, pe, in, idn, ie );
```

```
    return root;
```



node Construct (int Pre[], int PS, int PE, int In[], int IS, int IE) {

1 if (PS > PE || IS > IE) { return null; }

node root = new Node (Pre[PS]);

int idn = 0;

for (int i = IS; i <= IE; i++) {

2 if (In[i] == Pre[PS]) {
idn = i; break;

int Count = idn - IS;

3 root.left = Construct (Pre, PS+1, PS+Count, In, IS, idn);

4 root.right = Construct (Pre, PS+Count+1, PE, In, idn+1, IE);

5 return root;

Preorder[]: -11 -4 -3 -6 7 -2 3
Inorder[]: -3 -4 -6 -11 -2 7 3



Preorder[]: -11 -4 -3 -6 7 -2 3
Inorder[]: -3 -4 -6 -11 -2 7 3



Preorder[]: -11 -4 -3 -6 7 -2 3
Inorder[]: -3 -4 -6 -11 -2 7 3



Preorder[]: -11 -4 -3 -6 7 -2 3
Inorder[]: -3 -4 -6 -11 -2 7 3



Preorder[]: -11 -4 -3 -6 7 -2 3
Inorder[]: -3 -4 -6 -11 -2 7 3

